

49-production-ready-code

Modul 49: Production-Ready Code



Figure 1: Production-Ready Code

Level: Advanced

Prasyarat: Modul 6 (Express.js Dasar)

Output Target: Aplikasi Express.js production-ready dengan input validation, security hardening, health checks, dan environment configuration

Deskripsi

Modul ini membahas praktik mengubah aplikasi Express.js dari prototype menjadi production-ready. Fokus pada empat pilar utama:

validasi input dengan tipe aman, pengamanan aplikasi dari serangan umum, monitoring dengan health checks dan graceful shutdown, serta konfigurasi yang benar untuk berbagai environment.

Tabel Sesi

Sesi	Topik	Durasi	Target Pembelajaran
1	Input Validation & Error Handling	3 JP	Zod schema, type-safe validation, AppError class, error middleware, format respons error konsisten
2	Security Hardening	3 JP	Helmet, CORS, rate limiting, input sanitization, dependency audit
3	Health Checks & Graceful Shutdown	3 JP	/healthz, /readyz, SIGTERM handler, connection draining, timeout & circuit breaker, PM2
4	Environment & Configuration	3 JP	12-factor app, env validation, config per environment, secrets management, logging level

Referensi

- [Zod Documentation](#)
- [Helmet.js](#)
- [express-rate-limit](#)
- [PM2 Documentation](#)
- [12-Factor App](#)

Sesi 1: Input Validation & Error Handling

Durasi: 3 JP (135 menit)

Tujuan Pembelajaran

Setelah sesi ini, peserta mampu:

- Membuat Zod schema untuk validasi data input
- Menggunakan type inference dari Zod schema
- Membuat custom error class (AppError) dengan kode error, status code, dan metadata
- Membangun Express global error handler middleware
- Mengembalikan format respons error yang konsisten

1.1 Zod Schema Dasar

1.1.1 String & Number

```
import { z } from 'zod';

// Validasi string dasar
const usernameSchema = z.string().min(3).max(30);
const emailSchema = z.string().email();

// Validasi number
const ageSchema = z.number().int().positive().max(150);
const priceSchema = z.number().positive();
```

Method umum:

Method	Deskripsi	Contoh
.min(n)	Minimum length/value	z.string().min(3)
.max(n)	Maximum length/value	z.string().max(255)
.email()	Format email valid	z.string().email()
.url()	Format URL valid	z.string().url()
.int()	Bilangan bulat	z.number().int()
.positive()	Nilai positif	z.number().positive()
.optional()	Boleh undefined	z.string().optional()
.nullable()	Boleh null	z.string().nullable()

1.1.2 Object & Array

```
// Object schema
const userSchema = z.object({
  id: z.string().uuid(),
  username: z.string().min(3).max(30),
  email: z.string().email(),
  age: z.number().int().positive().optional(),
  role: z.enum(['admin', 'user', 'guest']),
});

// Array schema
const tagsSchema = z.array(z.string().min(1).max(10));
const scoresSchema = z.array(z.number().min(0).max(100));
```

1.1.3 Union & Discriminated Union

```
// Union – salah satu dari beberapa tipe
const resultSchema = z.union([
  z.object({ status: z.literal('success'), data: z.any() }),
  z.object({ status: z.literal('error'), message: z.string() }),
]);

// Discriminated Union – lebih efisien, pakai field pembeda
const apiResponseSchema = z.discriminatedUnion('status', [
  z.object({ status: z.literal('success'), data: z.any() }),
  z.object({ status: z.literal('error'), message: z.string() }),
  z.object({ status: z.literal('loading') }),
]);
```

1.1.4 Transform

```
// Transform nilai setelah validasi
const trimmedString = z.string().transform(s => s.trim());
const lowerCaseEmail = z.string().email().transform(s => s.toLowerCase());
const dateStringToDate = z.string().pipe(z.coerce.date());
const slugSchema = z.string().transform(s => s.toLowerCase().replace(/\s+/g, '-'));
```

1.2 Type-Safe Validation

Zod bisa meng-infer tipe TypeScript langsung dari schema.

```
import { z } from 'zod';

// Definisikan schema
const createUserSchema = z.object({
  username: z.string().min(3).max(30),
  email: z.string().email(),
  password: z.string().min(8),
});

// Infer tipe dari schema
type CreateUserInput = z.infer<typeof createUserSchema>;
// Hasil: { username: string; email: string; password: string }

// Gunakan di handler
function createUser(data: unknown) {
  const parsed = createUserSchema.parse(data);
  // parsed sekarang bertipe CreateUserInput – aman!
```

```
    return db.users.create({ data: parsed });
}
```

Validasi di middleware Express:

```
// validation.middleware.ts
import { Request, Response, NextFunction } from 'express';
import { ZodSchema } from 'zod';
import { AppError } from '../AppError';

export function validate(schema: ZodSchema, source: 'body' | 'query' | 'params') {
  return (req: Request, _res: Response, next: NextFunction) => {
    const result = schema.safeParse(req[source]);

    if (!result.success) {
      const details = result.error.issues.map(issue => ({
        field: issue.path.join('.'),
        message: issue.message,
      }));

      throw new AppError(400, 'VALIDATION_ERROR', 'Data tidak valid', details);
    }

    req[source] = result.data;
    next();
  };
}
```

1.3 Custom Error: AppError Class

Buat error class kustom untuk membawa informasi tambahan.

```
// AppError.ts
export class AppError extends Error {
  public readonly statusCode: number;
  public readonly code: string;
  public readonly details?: unknown;
  public readonly timestamp: Date;

  constructor(
    statusCode: number,
    code: string,
    message: string,
    details?: unknown,
  ) {
    super(message);
  }
}
```

```

    super(message);
    this.statusCode = statusCode;
    this.code = code;
    this.details = details;
    this.timestamp = new Date();

    // Maintain proper prototype chain
    Object.setPrototypeOf(this, AppError.prototype);
  }
}

```

Error code standar:

Kode	Status	Deskripsi
VALIDATION_ERROR	400	Data input tidak valid
UNAUTHORIZED	401	Belum terautentikasi
FORBIDDEN	403	Tidak punya akses
NOT_FOUND	404	Resource tidak ditemukan
CONFLICT	409	Konflik data (duplikat)
RATE_LIMIT	429	Terlalu banyak request
INTERNAL_ERROR	500	Error server internal

1.4 Error Middleware Express

Global error handler — satu tempat untuk semua error.

```

// errorHandler.middleware.ts
import { Request, Response, NextFunction } from 'express';
import { AppError } from '../AppError';
import { ZodError } from 'zod';

export function errorHandler(
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction,
) {
  // Log error untuk debugging
  console.error(`[${new Date().toISOString()}] Error:`, {
    name: err.name,
    message: err.message,
    stack: process.env.NODE_ENV === 'development' ? err.stack : undefined,
  });
}

```

```

// 1. AppError – error yang sudah kita definisikan
if (err instanceof AppError) {
  return res.status(err.statusCode).json({
    success: false,
    error: {
      code: err.code,
      message: err.message,
      details: err.details,
      timestamp: err.timestamp,
    },
  });
}

// 2. ZodError – error validasi dari Zod
if (err instanceof ZodError) {
  const details = err.issues.map(issue => ({
    field: issue.path.join('.'),
    message: issue.message,
  }));

  return res.status(400).json({
    success: false,
    error: {
      code: 'VALIDATION_ERROR',
      message: 'Data input tidak valid',
      details,
      timestamp: new Date(),
    },
  });
}

// 3. Error tak terduga
return res.status(500).json({
  success: false,
  error: {
    code: 'INTERNAL_ERROR',
    message: 'Terjadi kesalahan internal server',
    timestamp: new Date(),
  },
});
}

```

1.5 Format Respons Error Konsisten

Semua endpoint harus mengembalikan format error yang sama.

Format sukses:

```
{
  "success": true,
  "data": { ... }
}
```

Format error:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Data input tidak valid",
    "details": [
      { "field": "email", "message": "Email tidak valid" },
      { "field": "age", "message": "Usia harus positif" }
    ],
    "timestamp": "2025-07-05T10:30:00.000Z"
  }
}
```

1.6 Praktik: Integrasi Express + Zod + Error Handler

```
// app.ts
import express from 'express';
import { z } from 'zod';
import { AppError } from './AppError';
import { validate } from './validation.middleware';
import { errorHandler } from './errorHandler.middleware';

const app = express();
app.use(express.json());

// Schema
const registerSchema = z.object({
  username: z.string().min(3).max(30),
  email: z.string().email(),
  password: z.string().min(8).max(128),
});
```



```

// Route
app.post(
  '/api/register',
  validate(registerSchema),
  async (req, res) => {
    // Data sudah tervalidasi dan typed
    const { username, email, password } = req.body;

    // Simulasi duplikat
    if (email === 'exists@test.com') {
      throw new AppError(409, 'CONFLICT', 'Email sudah terdaftar');
    }

    res.status(201).json({
      success: true,
      data: { username, email },
    });
  },
);

// Global error handler – HARUS setelah semua route
app.use(errorHandler);

app.listen(3000, () => console.log('Server running on port 3000'));

```

Latihan

Soal 1: Zod Schema Produk

Buat Zod schema untuk input produk dengan spesifikasi:

1. name — string, min 3, max 100
2. price — number, positive, max 10000000
3. category — enum: 'food', 'electronics', 'clothing'
4. tags — array string, min 1, max 5
5. description — string optional, max 500
6. discount — number optional, antara 0 dan 100

Infer tipe CreateProductInput dari schema.

Soal 2: Error Middleware

Buat global error handler Express yang:

1. Tangani AppError — kembalikan format { success, error: { code, message, details } }

2. Tangani error `ZodError` — map ke `VALIDATION_ERROR` dengan field-level details
3. Tangani error lain — kembalikan `INTERNAL_ERROR`, jangan bocorkan detail di production
4. Log error dengan timestamp

Soal 3: Integrasi

Gabungkan Zod schema produk dan error handler dalam satu aplikasi Express.

- `POST /api/products` — validasi body dengan Zod, simpan (simulasi), return 201
- `POST /api/products/bulk` — validasi array produk, return 201
- Semua error pakai format konsisten

Pengantar Express Global Error Handler

Express middleware error memiliki 4 parameter: (`err`, `req`, `res`, `next`). Jika `next(err)` dipanggil atau error dilempar dalam handler async, Express akan mengeksekusi middleware error terdaftar.

// Wajib: 4 parameter!

```
app.use((err: Error, req: Request, res: Response, next: NextFunction) => {  
  // handle error  
});
```

Peringatan: Middleware error harus didaftarkan **setelah** semua route. Jika didaftarkan sebelum route, error handler tidak akan pernah terpanggil.

Sesi 2: Security Hardening

Durasi: 3 JP (135 menit)

Tujuan Pembelajaran

Setelah sesi ini, peserta mampu:

- Mengimplementasikan Helmet untuk keamanan header HTTP
- Mengonfigurasi CORS dengan benar
- Menerapkan rate limiting dengan `express-rate-limit`
- Membersihkan input dari serangan XSS dan SQL injection
- Melakukan audit keamanan dependency

2.1 Helmet

Helmet mengamankan aplikasi Express dengan mengatur berbagai header HTTP.

Instalasi

```
npm install helmet
```

Konfigurasi Dasar

```
import helmet from 'helmet';
import express from 'express';

const app = express();

// Gunakan semua middleware Helmet default
app.use(helmet());
```

Header yang Diatur Helmet

Header	Default	Fungsi
Content-Security-Policy	default-src 'self'	Mencegah XSS dengan membatasi sumber konten
X-Content-Type-Options	nosniff	Mencegah MIME sniffing
X-Frame-Options	SAMEORIGIN	Mencegah clickjacking
X-XSS-Protection	0	Nonaktifkan XSS filter lama browser
Strict-Transport-Security	max-age=15552000	Memaksa HTTPS
Referrer-Policy	no-referrer	Mengontrol informasi referer

Konfigurasi Lanjutan

```
app.use(
  helmet({
    // Content Security Policy kustom
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],

```

```

        scriptSrc: ['self', 'unsafe-inline', 'cdn.example.com'],
        styleSrc: ['self', 'unsafe-inline'],
        imgSrc: ['self', 'data:', 'images.example.com'],
        connectSrc: ['self', 'api.example.com'],
        fontSrc: ['self', 'fonts.googleapis.com'],
        objectSrc: ['none'],
        upgradeInsecureRequests: [],
    },
},

// Frame guard – siapa yang boleh membungkus dalam iframe
frameguard: {
    action: 'deny', // 'deny' | 'sameorigin' | 'allow-from'
},

// HSTS – HTTP Strict Transport Security
hsts: {
    maxAge: 31536000, // 1 tahun dalam detik
    includeSubDomains: true,
    preload: true,
},

// Nonaktifkan header yang tidak diperlukan
xPoweredBy: false, // Hapus header X-Powered-By
}),
);

```

CSP (Content Security Policy) — Penjelasan

CSP mencegah XSS dengan mendeklarasikan sumber konten yang sah.

Content-Security-Policy: default-src 'self'; script-src 'self' cdn.example.com; img-src 'self' data:

- default-src 'self' — semua konten hanya dari domain sendiri
- script-src — sumber script yang diizinkan
- img-src — sumber gambar yang diizinkan
- 'unsafe-inline' — izinkan script inline (kurangi keamanan, kadang diperlukan)

2.2 CORS (Cross-Origin Resource Sharing)

CORS mengontrol domain mana yang bisa mengakses API.

Instalasi

```
npm install cors
npm install -D @types/cors
```

Konfigurasi Dasar

```
import cors from 'cors';
import express from 'express';

const app = express();

// Izinkan semua origin (tidak untuk production!)
app.use(cors());
```

Konfigurasi Production

```
const allowedOrigins = [
  'https://app.example.com',
  'https://admin.example.com',
  /\.example\.com$/, // Regex untuk subdomain
];

app.use(
  cors({
    origin: (origin, callback) => {
      // Izinkan request tanpa origin (Postman, server-side)
      if (!origin) return callback(null, true);

      if (allowedOrigins.some(allowed =>
        allowed instanceof RegExp
          ? allowed.test(origin)
          : allowed === origin
      )) {
        callback(null, true);
      } else {
        callback(new Error('Tidak diizinkan oleh CORS'));
      }
    },
    credentials: true, // Izinkan cookie/Authorization header
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH'],
    allowedHeaders: ['Content-Type', 'Authorization'],
    exposedHeaders: ['X-Total-Count', 'X-Rate-Limit'],
    maxAge: 86400, // Cache preflight 24 jam (dalam detik)
  }),
);
```

CORS Preflight

Browser mengirim request OPTIONS sebelum request kompleks (method selain GET/HEAD/POST, atau dengan custom header).

```
// CORS middleware otomatis menangani preflight
// Tapi bisa dikustomisasi:
app.options('/api/*', cors()); // Handle preflight untuk semua route API
```

2.3 Rate Limiting

Mencegah abuse dengan membatasi jumlah request per waktu.

Instalasi

```
npm install express-rate-limit
```

Konfigurasi Dasar

```
import rateLimit from 'express-rate-limit';

// Global rate limit - semua route
const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 menit
  max: 100,                 // maks 100 request per window
  standardHeaders: true,    // Return rate limit info di header
  legacyHeaders: false,     // Nonaktifkan header X-RateLimit-*
  message: {
    success: false,
    error: {
      code: 'RATE_LIMIT',
      message: 'Terlalu banyak request, coba lagi nanti',
    },
  },
});

app.use(globalLimiter);
```

Sliding Window

Secara default express-rate-limit menggunakan fixed window. Untuk sliding window:

```
import rateLimit from 'express-rate-limit';
```

```
const slidingLimiter = rateLimit({
  windowMs: 60 * 1000,    // 1 menit
  max: 30,
  // sliding window dengan mencatat timestamp per request
  // express-rate-limit v7+ menggunakan sliding window secara default
  standardHeaders: true,
  legacyHeaders: false,
});
```

Sliding window membagi jendela menjadi segmen-segmen kecil. Saat request masuk, sistem menghitung jumlah request dalam rentang waktu yang benar-benar bergulir, bukan reset per menit penuh.

Per-Route vs Global

```
// Global limiter – lemah, untuk semua endpoint
app.use(globalLimiter);

// Per-route limiter – ketat, untuk endpoint sensitif
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 menit
  max: 5,                  // hanya 5 percobaan login
  message: {
    success: false,
    error: {
      code: 'RATE_LIMIT',
      message: 'Terlalu banyak percobaan login',
    },
  },
});

app.post('/api/login', authLimiter, loginHandler);
app.post('/api/register', authLimiter, registerHandler);

// API limiter – untuk endpoint umum
const apiLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 menit
  max: 60,
});

app.use('/api/', apiLimiter);
```

2.4 Input Sanitization

Membersihkan input untuk mencegah XSS dan injection.

XSS Prevention

```
// Menggunakan xss (npm package)
import xss from 'xss';

function sanitizeInput(input: string): string {
  return xss(input, {
    whitelist: {}, // Tidak ada tag HTML yang diizinkan
    stripIgnoreTag: true, // Hapus tag yang tidak dikenal
    stripIgnoreTagBody: ['script', 'style'], // Hapus konten tag berbahaya
  });
}

// Middleware sanitasi
function sanitizeBody(req: Request, _res: Response, next: NextFunction) {
  if (req.body) {
    for (const key in req.body) {
      if (typeof req.body[key] === 'string') {
        req.body[key] = sanitizeInput(req.body[key]);
      }
    }
  }
  next();
}
```

DOMPurify (untuk konten HTML yang diizinkan)

npm install dompurify jsdom

```
import { JSDOM } from 'jsdom';
import DOMPurify from 'dompurify';

const window = new JSDOM('').window;
const purify = DOMPurify(window as any);

function sanitizeHtml(html: string): string {
  return purify.sanitize(html, {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br'],
    ALLOWED_ATTR: ['href', 'target'],
  });
}
```



```
// Input: <script>alert('xss')</script><p>Hello <b>World</b></p>
// Output: <p>Hello <b>World</b></p>
```

SQL Injection Prevention

Di Node.js, SQL injection dicegah dengan **parameterized query / prepared statement**.

```
// □ RENTAN – concatenation
const query = `SELECT * FROM users WHERE email = '${email}'`;

// □ AMAN – parameterized query (mysql2)
const query = 'SELECT * FROM users WHERE email = ?';
const [rows] = await connection.execute(query, [email]);

// □ AMAN – parameterized query (pg / node-postgres)
const query = 'SELECT * FROM users WHERE email = $1';
const result = await client.query(query, [email]);

// □ AMAN – Prisma ORM (prepared statement otomatis)
const user = await prisma.user.findUnique({ where: { email } });
```

Aturan emas: Jangan pernah menggabungkan input user langsung ke query SQL. Selalu gunakan placeholder (? atau \$1).

Validasi + Sanitasi Bersamaan

```
import { z } from 'zod';

const commentSchema = z.object({
  title: z.string().min(1).max(100),
  content: z.string().min(1).max(1000),
  authorName: z.string().min(1).max(50),
});

function sanitizeAndValidate(data: unknown) {
  const sanitized = {
    title: xss(data.title),
    content: xss(data.content),
    authorName: xss(data.authorName),
  };
  return commentSchema.parse(sanitized);
}
```

2.5 Dependency Security

npm audit

Audit dependency untuk vulnerability

npm audit

Perbaiki otomatis (patch version)

npm audit fix

Lihat detail

npm audit --json

Snyk

Instalasi

npm install -g snyk

Auth

snyk auth

Test project

snyk test

Monitor (pantau terus)

snyk monitor

Lock File

Selalu commit lock file!

package-lock.json – npm

yarn.lock – yarn

pnpm-lock.yaml – pnpm

Lock file memastikan:

1. Semua developer install versi dependency yang sama

2. CI/CD menggunakan versi yang sudah terverifikasi

3. Rollback lebih mudah

Semantic Version Pinning

```
{
  "dependencies": {
    "express": "^4.18.2",    // Izinkan minor/patch update
    "helmet": "7.1.0",      // Pin exact version
    "zod": "~3.22.0",       // Hanya patch update
  }
}
```

```

    "cors": "^2.8.5"
  }
}

```

Prefix	Makna	Rentang
^	Compatible	>=3.22.0 <4.0.0
~	Approximate	>=3.22.0 <3.23.0
no prefix	Exact	3.22.0 saja

Checklist Keamanan Dependency

- ☐ Lock file di-commit
- ☐ npm audit jalan di CI
- ☐ Dependency tidak deprecated
- ☐ Versi pin untuk dependency kritis
- ☐ Regular update (bulanan)
- ☐ Hapus dependency tidak terpakai (npm prune)

2.6 Praktik Lengkap

```

import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import rateLimit from 'express-rate-limit';
import xss from 'xss';

const app = express();

// 1. Helmet – keamanan header
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'"],
      imgSrc: ["'self'", 'data:'],
    },
  },
}));

// 2. CORS – akses terkontrol
app.use(cors({
  origin: ['https://app.example.com'],

```

```

    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
  }));

// 3. Body parser – sebelum rate limiter
app.use(express.json({ limit: '10kb' }));

// 4. Sanitasi input
app.use((req, _res, next) => {
  if (req.body) {
    for (const key in req.body) {
      if (typeof req.body[key] === 'string') {
        req.body[key] = xss(req.body[key]);
      }
    }
  }
  next();
});

// 5. Rate limiting global
app.use(rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
}));

// Route
app.post('/api/comment', (req, res) => {
  res.json({ success: true, data: req.body });
});

app.listen(3000);

```

Latihan

Soal 1: Implementasi Helmet

Buat konfigurasi Helmet untuk aplikasi Express dengan:

1. CSP: script hanya dari 'self' dan cdn.trusted.com, style dari 'self' dan fonts.googleapis.com
2. Frame guard: deny semua
3. HSTS: 1 tahun, include subdomains, preload
4. Nonaktifkan header X-Powered-By

Soal 2: CORS Konfigurasi

Buat CORS configuration yang:

1. Izinkan origin: `https://dashboard.example.com` dan `https://api.example.com`
2. Izinkan credentials (cookie)
3. Izinkan method: GET, POST, PUT, PATCH, DELETE
4. Cache preflight 1 jam
5. Kembalikan error jika origin tidak dikenal

Soal 3: Rate Limiting

Buat rate limiter untuk aplikasi e-commerce:

1. Global: 100 request/menit
2. Auth routes (`/api/auth/*`): 5 request/menit untuk login, 10 request/menit untuk register
3. API routes (`/api/*`): 60 request/menit
4. Format error konsisten dengan `{ success, error: { code, message } }`

Sesi 3: Health Checks & Graceful Shutdown

Durasi: 3 JP (135 menit)

Tujuan Pembelajaran

Setelah sesi ini, peserta mampu:

- Membuat endpoint `/healthz` (liveness) dan `/readyz` (readiness)
- Mengecek kesehatan database, Redis, dan service eksternal
- Mengimplementasikan graceful shutdown dengan SIGTERM handler
- Menggunakan timeout dan circuit breaker
- Mengelola aplikasi dengan PM2 ecosystem

3.1 Health Check Endpoints

Health check dibagi dua jenis: **liveness** dan **readiness**.

Liveness (`/healthz`)

Apakah aplikasi masih hidup? Cek sederhana — server merespons.

```
// health.controller.ts
import { Request, Response } from 'express';

export function healthz(_req: Request, res: Response) {
  res.status(200).json({
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
  });
}
```

Readiness (/readyz)

Apakah aplikasi siap menerima traffic? Cek dependency — DB, Redis, dll.

```
// readyz handler
interface HealthCheckResult {
  service: string;
  status: 'healthy' | 'unhealthy';
  latency: number;
  error?: string;
}

async function checkDatabase(): Promise<HealthCheckResult> {
  const start = Date.now();
  try {
    await db.$queryRaw`SELECT 1`; // Prisma
    return {
      service: 'database',
      status: 'healthy',
      latency: Date.now() - start,
    };
  } catch (err) {
    return {
      service: 'database',
      status: 'unhealthy',
      latency: Date.now() - start,
      error: err instanceof Error ? err.message : 'Unknown error',
    };
  }
}

async function checkRedis(): Promise<HealthCheckResult> {
  const start = Date.now();
  try {
```

```

    await redis.ping();
    return {
      service: 'redis',
      status: 'healthy',
      latency: Date.now() - start,
    };
  } catch (err) {
    return {
      service: 'redis',
      status: 'unhealthy',
      latency: Date.now() - start,
      error: err instanceof Error ? err.message : 'Unknown error',
    };
  }
}

async function checkExternalService(url: string): Promise<HealthCheckResult> {
  const start = Date.now();
  try {
    const response = await fetch(url, {
      method: 'GET',
      signal: AbortSignal.timeout(5000), // timeout 5 detik
    });
    return {
      service: url,
      status: response.ok ? 'healthy' : 'unhealthy',
      latency: Date.now() - start,
    };
  } catch (err) {
    return {
      service: url,
      status: 'unhealthy',
      latency: Date.now() - start,
      error: err instanceof Error ? err.message : 'Unknown error',
    };
  }
}

```

Handler Readiness Terintegrasi

```

// readyz handler
export async function readyz(_req: Request, res: Response) {
  const checks = await Promise.all([
    checkDatabase(),
    checkRedis(),
  ]);
}

```

```

    checkExternalService('https://api.payment.com/health'),
  ]);

  const allHealthy = checks.every(c => c.status === 'healthy');
  const statusCode = allHealthy ? 200 : 503;

  res.status(statusCode).json({
    status: allHealthy ? 'ok' : 'degraded',
    timestamp: new Date().toISOString(),
    checks,
  });
}

```

Contoh respons:

```

{
  "status": "ok",
  "timestamp": "2025-07-05T10:30:00.000Z",
  "checks": [
    { "service": "database", "status": "healthy", "latency": 5 },
    { "service": "redis", "status": "healthy", "latency": 2 },
    { "service": "https://api.payment.com/health", "status": "healthy", "latency": 10 }
  ]
}

```

Routing Health Check

```

import { Router } from 'express';

const healthRouter = Router();

// Liveness – cepat, tidak perlu cek dependency
healthRouter.get('/healthz', healthz);

// Readiness – cek semua dependency
healthRouter.get('/readyz', readyz);

// Gabungan
healthRouter.get('/health', async (_req, res) => {
  const [liveness, readiness] = await Promise.all([
    Promise.resolve({ status: 'ok' }),
    checkAllServices(),
  ]);

  res.status(liveness.status === 'ok' ? 200 : 503).json({
    ...liveness,
  });
});

```



```

        readiness,
    });
});

export { healthRouter };

```

3.2 Graceful Shutdown

Saat aplikasi menerima sinyal berhenti (SIGTERM), kita harus: 1. Berhenti menerima request baru 2. Selesaikan request yang sedang berjalan 3. Tutup koneksi database, Redis, dll. 4. Hapus resource sementara

```

// graceful-shutdown.ts
import { Server } from 'http';
import { prisma } from './db';
import { redis } from './redis';

export function setupGracefulShutdown(server: Server) {
    let isShuttingDown = false;

    function shutdown(signal: string) {
        return async () => {
            if (isShuttingDown) return;
            isShuttingDown = true;

            console.log(`[${signal}] Memulai graceful shutdown...`);

            // 1. Set health check ke tidak sehat – orchestrator tahu untuk stop routing
            // (biasanya di-set di shared state atau env variable)

            // 2. Stop menerima request baru
            server.close(async (err) => {
                if (err) {
                    console.error('Error saat menutup server:', err);
                    process.exit(1);
                }

                console.log('Server HTTP ditutup');

                // 3. Tutup koneksi database
                try {
                    await prisma.$disconnect();
                    console.log('Koneksi database ditutup');
                }
            });
        };
    }
}

```

```

    } catch (err) {
      console.error('Error menutup database:', err);
    }

    // 4. Tutup koneksi Redis
    try {
      await redis.quit();
      console.log('Koneksi Redis ditutup');
    } catch (err) {
      console.error('Error menutup Redis:', err);
    }

    // 5. Cleanup task lain
    // - Hapus file temporary
    // - Flush log buffer
    // - Cancel job queue

    console.log('Graceful shutdown selesai');
    process.exit(0);
  });

  // 6. Force shutdown jika melebihi batas waktu
  setTimeout(() => {
    console.error('Force shutdown – timeout 30 detik tercapai');
    process.exit(1);
  }, 30_000).unref(); // .unref() agar tidak menghalangi proses keluar
}

// Tangani berbagai sinyal
process.on('SIGTERM', shutdown('SIGTERM')); // Docker/k8s mengirim ini
process.on('SIGINT', shutdown('SIGINT')); // Ctrl+C
process.on('SIGQUIT', shutdown('SIGQUIT')); // Ctrl+\

// Tangani uncaught exception – crash, tapi tetap cleanup
process.on('uncaughtException', (err) => {
  console.error('Uncaught exception:', err);
  shutdown('UNCAUGHT_EXCEPTION')();
});

// Tangani unhandled rejection – log saja
process.on('unhandledRejection', (reason) => {
  console.error('Unhandled rejection:', reason);
});
}

```

Implementasi di Aplikasi

```
// app.ts
import express from 'express';
import { createServer } from 'http';
import { healthRouter } from './health.controller';
import { setupGracefulShutdown } from './graceful-shutdown';

const app = express();
const server = createServer(app);

// Route
app.use('/api/health', healthRouter);
app.get('/api/users', (req, res) => { /* ... */ });

// Setup graceful shutdown
setupGracefulShutdown(server);

// Start server
const PORT = process.env.PORT || 3000;
server.listen(PORT, () => {
  console.log(`Server berjalan di port ${PORT}`);
});
```

Connection Draining

Connection draining memastikan request in-flight selesai sebelum shutdown.

```
// Middleware untuk menolak request baru saat shutdown
let isShuttingDown = false;

const shutdownMiddleware = (req: Request, res: Response, next: NextFunction) => {
  if (isShuttingDown) {
    res.status(503).json({
      success: false,
      error: {
        code: 'SERVICE_UNAVAILABLE',
        message: 'Server sedang dalam maintenance',
      },
    });
    return;
  }
  next();
};
```

```
app.use(shutdownMiddleware);
```

3.3 Timeout & Circuit Breaker

Server Timeout

```
import { createServer } from 'http';

const server = createServer(app);

// Timeout untuk merespons (dalam milidetik)
server.timeout = 30_000; // 30 detik - timeout default
server.headersTimeout = 60_000; // 60 detik - timeout header
server.requestTimeout = 30_000; // 30 detik - timeout request
server.keepAliveTimeout = 5_000; // 5 detik - keep alive

// Atau di Express
app.use((req, res, next) => {
  // Set timeout per request
  req.setTimeout(30_000, () => {
    res.status(408).json({
      success: false,
      error: {
        code: 'REQUEST_TIMEOUT',
        message: 'Request timeout',
      },
    });
    req.destroy();
  });
  next();
});
```

Retry dengan Backoff

```
// retry.ts
interface RetryConfig {
  maxRetries: number;
  baseDelay: number; // dalam ms
  maxDelay: number; // dalam ms
  factor: number; // multiplier
}

async function retryWithBackoff<T>(
  fn: () => Promise<T>,
```

```

config: RetryConfig = {
    maxRetries: 3,
    baseDelay: 1000,
    maxDelay: 10_000,
    factor: 2,
},
): Promise<T> {
    let lastError: Error | undefined;

    for (let attempt = 1; attempt <= config.maxRetries; attempt++) {
        try {
            return await fn();
        } catch (err) {
            lastError = err instanceof Error ? err : new Error(String(err));

            if (attempt === config.maxRetries) break;

            // Exponential backoff with jitter
            const delay = Math.min(
                config.baseDelay * Math.pow(config.factor, attempt - 1),
                config.maxDelay,
            );
            const jitter = Math.random() * 1000; // acak 0-1000ms

            console.warn(
                `Percobaan ${attempt}/${config.maxRetries} gagal. ` +
                `Retry dalam ${(delay + jitter) / 1000} detik...`
            );

            await new Promise(resolve => setTimeout(resolve, delay + jitter));
        }
    }

    throw lastError!;
}

// Penggunaan
async function fetchPaymentData(orderId: string) {
    return retryWithBackoff(() =>
        fetch(`https://payment-api.example.com/orders/${orderId}`).then(r => {
            if (!r.ok) throw new Error(`HTTP ${r.status}`);
            return r.json();
        })
    );
}

```

Circuit Breaker

```
// circuit-breaker.ts
class CircuitBreaker {
  private failures = 0;
  private lastFailureTime = 0;
  private state: 'closed' | 'open' | 'half-open' = 'closed';

  constructor(
    private threshold: number = 5,
    private resetTimeout: number = 30_000, // 30 detik
  ) {}

  async call<T>(fn: () => Promise<T>): Promise<T> {
    if (this.state === 'open') {
      // Cek apakah sudah waktunya half-open
      if (Date.now() - this.lastFailureTime >= this.resetTimeout) {
        this.state = 'half-open';
      } else {
        throw new Error('Circuit breaker open');
      }
    }

    try {
      const result = await fn();
      // Sukses - reset
      if (this.state === 'half-open') {
        this.state = 'closed';
        this.failures = 0;
      }
      return result;
    } catch (err) {
      this.failures++;
      this.lastFailureTime = Date.now();

      if (this.failures >= this.threshold) {
        this.state = 'open';
      }

      throw err;
    }
  }

  getState() {
    return this.state;
  }
}
```

```

    reset() {
      this.failures = 0;
      this.state = 'closed';
    }
  }

  // Penggunaan
  const paymentBreaker = new CircuitBreaker(3, 10_000);

  app.get('/api/payment/:id', async (req, res) => {
    try {
      const data = await paymentBreaker.call(() =>
        fetchPaymentData(req.params.id)
      );
      res.json({ success: true, data });
    } catch (err) {
      if (err instanceof Error && err.message === 'Circuit breaker open') {
        res.status(503).json({
          success: false,
          error: {
            code: 'SERVICE_UNAVAILABLE',
            message: 'Layanan pembayaran sedang tidak tersedia. Coba lagi nanti.',
          },
        });
      } else {
        throw err;
      }
    }
  });
});

```

3.4 PM2 Ecosystem

PM2 — production process manager untuk Node.js.

Instalasi

```
npm install -g pm2
```

Ecosystem File

```

// ecosystem.config.js
module.exports = {

```

```

apps: [{
  name: 'my-app',
  script: 'dist/app.js',

  // Cluster mode – jalankan di semua CPU
  instances: 'max',      // Gunakan semua CPU core
  exec_mode: 'cluster',  // 'cluster' atau 'fork'

  // Watch & reload
  watch: ['dist'],      // Watch folder untuk auto restart
  ignore_watch: ['node_modules', 'logs'],
  watch_delay: 1000,

  // Memory
  max_memory_restart: '500M', // Restart jika memori > 500MB

  // Logging
  error_file: './logs/error.log',
  out_file: './logs/out.log',
  log_file: './logs/combined.log',
  merge_logs: true,
  log_date_format: 'YYYY-MM-DD HH:mm:ss',

  // Environment
  env: {
    NODE_ENV: 'production',
    PORT: 3000,
  },
  env_staging: {
    NODE_ENV: 'staging',
    PORT: 3001,
  },

  // Graceful shutdown
  kill_timeout: 30_000,      // Max 30 detik untuk shutdown
  listen_timeout: 10_000,    // Tunggu 10 detik untuk ready
  shutdown_with_message: true, // Kirim pesan shutdown ke app

  // Restart strategy
  min_uptime: '10s',        // Minimal uptime untuk dianggap stabil
  max_restarts: 10,         // Max restart dalam min_uptime
  restart_delay: 5000,      // Delay antar restart
}],
};

```


PM2 Commands

Start aplikasi

```
pm2 start ecosystem.config.js
```

Start dengan environment spesifik

```
pm2 start ecosystem.config.js --env staging
```

Lihat status

```
pm2 status
```

```
pm2 list
```

```
pm2 show my-app
```

Monitor real-time

```
pm2 monit
```

Logs

```
pm2 logs
```

```
pm2 logs my-app --lines 100
```

Restart / Reload (zero-downtime)

```
pm2 restart my-app
```

```
pm2 reload my-app
```

Reload tanpa downtime (cluster mode)

```
pm2 gracefulReload my-app
```

Graceful reload

Stop / Delete

```
pm2 stop my-app
```

```
pm2 delete my-app
```

Startup script – auto restart saat server reboot

```
pm2 startup
```

```
pm2 save
```

Zero-Downtime Reload

Di cluster mode, PM2 melakukan reload dengan mengganti worker satu per satu:

```
pm2 reload my-app
```

Proses: 1. PM2 start worker baru 2. Worker baru listen ke port 3. Setelah worker baru siap, PM2 kirim SIGTERM ke worker lama 4. Worker lama selesaikan request in-flight lalu mati 5. Ulangi untuk setiap worker

PM2 + Graceful Shutdown

```
// Di kode aplikasi – handle PM2 shutdown message
process.on('message', (msg) => {
  if (msg === 'shutdown') {
    console.log('Menerima pesan shutdown dari PM2');
    // Mulai graceful shutdown
    shutdown('PM2_SHUTDOWN')();
  }
});
```

3.5 Praktik Lengkap

```
// server.ts
import express from 'express';
import { createServer } from 'http';

const app = express();
const server = createServer(app);

// 1. Health checks
app.get('/healthz', (_req, res) => {
  res.json({ status: 'ok', uptime: process.uptime() });
});

app.get('/readyz', async (_req, res) => {
  const dbOk = await checkDatabase();
  const status = dbOk ? 200 : 503;
  res.status(status).json({
    status: dbOk ? 'ok' : 'degraded',
    checks: [{ service: 'database', status: dbOk ? 'healthy' : 'unhealthy' }],
  });
});

// 2. Shutdown middleware
let shuttingDown = false;
app.use((_req, res, next) => {
  if (shuttingDown) {
    return res.status(503).json({ success: false, error: { code: 'MAINTENANCE' } });
  }
  next();
});

// 3. Request timeout
```

```

app.use((req, res, next) => {
  req.setTimeout(30_000, () => {
    res.status(408).json({ success: false, error: { code: 'TIMEOUT' } });
  });
  next();
});

// 4. Graceful shutdown handler
function shutdownHandler() {
  if (shuttingDown) return;
  shuttingDown = true;

  console.log('Graceful shutdown dimulai...');
  server.close(() => {
    console.log('Server ditutup');
    process.exit(0);
  });

  setTimeout(() => { process.exit(1); }, 30_000);
}

process.on('SIGTERM', shutdownHandler);
process.on('SIGINT', shutdownHandler);

server.listen(3000);

```

Latihan

Soal 1: Health Check Endpoint

Buat dua endpoint health check:

1. GET /healthz — liveness, return { status: 'ok', uptime, timestamp }
2. GET /readyz — readiness, cek koneksi database (simulasi) dan return { status, checks[] }

Database check simulasi: 80% chance healthy, 20% chance unhealthy.

Soal 2: Graceful Shutdown

Implementasikan graceful shutdown yang:

1. Tangani SIGTERM dan SIGINT

2. Set flag `isShuttingDown` untuk menolak request baru (return 503)
3. Tunggu server close (timeout 10 detik)
4. Log setiap tahap
5. Force shutdown jika melebihi timeout

Soal 3: PM2 Ecosystem

Buat `ecosystem.config.js` untuk aplikasi Express:

1. Cluster mode dengan semua CPU
2. Max memory restart 300MB
3. Watch folder `dist/`
4. Graceful shutdown dengan 20 detik timeout
5. Environment: production (PORT=3000) dan staging (PORT=3001)
6. Log error ke `logs/error.log`

Sesi 4: Environment & Configuration

Durasi: 3 JP (135 menit)

Tujuan Pembelajaran

Setelah sesi ini, peserta mampu:

- Menerapkan prinsip 12-factor app untuk konfigurasi
- Memvalidasi environment variables dengan `dotenv` dan `Zod`
- Mengelola konfigurasi per environment (`dev/staging/prod`)
- Mengelola secrets dengan aman
- Mengatur logging level dan feature flags berdasarkan environment

4.1 12-Factor App — Konfigurasi

12-factor app adalah metodologi untuk membangun aplikasi SaaS. Tiga faktor relevan dengan konfigurasi:

III. Config — Simpan konfigurasi di environment

Pendekatan yang benar: Simpan konfigurasi (yang bervariasi antar deployment) di **environment variables**, bukan di kode.

```
// ❌ BENAR — config dari env
const dbUrl = process.env.DATABASE_URL;
const port = Number(process.env.PORT) || 3000;
```

```
// ❌ SALAH – hardcode config
const dbUrl = 'postgresql://localhost:5432/myapp';
const port = 3000;
```

IV. Backing Services — Treat backing services as attached resources

Database, Redis, API eksternal adalah **attached resources**. Ganti URL-nya di env, bukan di kode.

```
// ✅ BENAR – ganti service lewat env
const services = {
  database: process.env.DATABASE_URL,
  redis: process.env.REDIS_URL,
  paymentApi: process.env.PAYMENT_API_URL,
};

// ❌ SALAH – hardcode URL
const services = {
  database: 'postgresql://localhost:5432/mydb',
  redis: 'redis://localhost:6379',
  paymentApi: 'https://api.payment.com',
};
```

V. Build, Release, Run — Pisahkan tahapan

Build: Compile kode → artifact
Release: Gabungkan artifact + config → release
Run: Jalankan release

```
Build → Release → Run
          ↑
        Config dari env
```

4.2 Env Validation dengan dotenv + Zod

Instalasi

```
npm install dotenv zod
```

File .env

```
# .env – jangan di-commit!
# copy ke .env.example untuk dokumentasi
```

```
PORT=3000
NODE_ENV=development
DATABASE_URL=postgresql://user:password@localhost:5432/mydb
REDIS_URL=redis://localhost:6379
JWT_SECRET=my-super-secret-key
LOG_LEVEL=debug
CORS_ORIGIN=http://localhost:5173
```

Zod Env Schema

```
// env.ts
import { z } from 'zod';
import dotenv from 'dotenv';

// Load .env file
dotenv.config();

// Schema validasi env
const envSchema = z.object({
  // App
  NODE_ENV: z.enum(['development', 'staging', 'production']).default('development'),
  PORT: z.coerce.number().int().positive().max(65535).default(3000),

  // Database
  DATABASE_URL: z.string().url(),
  DB_POOL_MIN: z.coerce.number().int().min(0).default(2),
  DB_POOL_MAX: z.coerce.number().int().min(1).max(100).default(10),

  // Redis
  REDIS_URL: z.string().url().optional(),

  // Auth
  JWT_SECRET: z.string().min(32, 'JWT_SECRET minimal 32 karakter'),
  JWT_EXPIRES_IN: z.string().default('7d'),

  // CORS
  CORS_ORIGIN: z.string().default('http://localhost:5173'),

  // Logging
  LOG_LEVEL: z.enum(['error', 'warn', 'info', 'debug']).default('info'),

  // Feature flags
  ENABLE_CACHE: z.coerce.boolean().default(true),
  ENABLE_RATE_LIMIT: z.coerce.boolean().default(true),
});
```

```

// Parse dan validasi
const parsedEnv = envSchema.safeParse(process.env);

if (!parsedEnv.success) {
  console.error('❌ Environment variables tidak valid:');
  for (const issue of parsedEnv.error.issues) {
    console.error(` - ${issue.path.join('.')}: ${issue.message}`);
  }
  process.exit(1);
}

// Export validated config
export const env = parsedEnv.data;

// Type dari schema
export type Env = z.infer<typeof envSchema>;

```

Penggunaan

```

// app.ts
import { env } from './env';

const app = express();

app.listen(env.PORT, () => {
  console.log(`Server berjalan di port ${env.PORT} (${env.NODE_ENV})`);
});

// Koneksi database
const db = new Pool({
  connectionString: env.DATABASE_URL,
  min: env.DB_POOL_MIN,
  max: env.DB_POOL_MAX,
});

```

.env.example

```

# .env.example – dokumentasi, di-commit ke repo
# Copy file ini ke .env dan isi nilainya

# ===== APP =====
# Environment: development | staging | production
NODE_ENV=development
# Port aplikasi

```

PORT=3000

```
# ===== DATABASE =====
# Contoh: postgresql://user:password@host:5432/dbname
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
# Pool koneksi (opsional, default: min=2, max=10)
DB_POOL_MIN=2
DB_POOL_MAX=10

# ===== REDIS =====
# Untuk caching dan session (opsional)
# REDIS_URL=redis://localhost:6379

# ===== AUTH =====
# JWT secret key – minimal 32 karakter
JWT_SECRET=your-secret-key-change-in-production
# JWT expiry (contoh: 7d, 1h, 30m)
JWT_EXPIRES_IN=7d

# ===== CORS =====
# Origin yang diizinkan
CORS_ORIGIN=http://localhost:5173

# ===== LOGGING =====
# Level: error | warn | info | debug
LOG_LEVEL=info

# ===== FEATURE FLAGS =====
ENABLE_CACHE=true
ENABLE_RATE_LIMIT=true
```

4.3 Config Per Environment

Struktur File

```
project/
├── .env.example      # Template untuk developer
├── .env.development  # Local development
├── .env.staging       # Staging server
├── .env.production   # Production (jangan di-commit!)
├── src/
│   └── env.ts        # Validasi Zod
```


Memilih File Berdasarkan NODE_ENV

```
// env.ts - enhanced version
import { z } from 'zod';
import dotenv from 'dotenv';
import path from 'path';

const nodeEnv = process.env.NODE_ENV || 'development';

// Load env file berdasarkan NODE_ENV
const envFile = path.resolve(process.cwd(), `.env.${nodeEnv}`);

// Fallback: .env.development → .env
dotenv.config({ path: path.resolve(process.cwd(), `.env.${nodeEnv}`) });
dotenv.config({ path: path.resolve(process.cwd(), '.env') }); // override dengan

// Validasi tetap sama
const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'staging', 'production']),
  PORT: z.coerce.number().int().positive().max(65535),
  DATABASE_URL: z.string().url(),
  // ... sisanya
});

const parsedEnv = envSchema.safeParse(process.env);
// ... error handling
export const env = parsedEnv!.data;
```

Perbedaan Config Per Environment

Config	Development	Staging	Production
PORT	3000	3000	80/443
DATABASE_URL	Local Postgres	Staging DB	Production DB
REDIS_URL	Local Redis	Staging Redis	Production Redis
JWT_SECRET	Dev-only	Staging-only	Real secret
LOG_LEVEL	debug	info	warn
CORS_ORIGIN	http://localhost:5173	http://staging-app.com	http://app.com
ENABLE_CACHE	false	true	true
ENABLE_RATE_LIMIT	false	true	true

4.4 Secrets Management

Jangan simpan secrets di kode atau di file `.env` yang di-commit.

Pendekatan Secrets Management

Metode	Keamanan	Use Case
<code>.env</code> file (lokal)	Rendah	Development
Environment variable di CI/CD	Sedang	CI/CD pipeline
Vault (HashiCorp)	Tinggi	Enterprise
Doppler	Tinggi	Team production
AWS Secrets Manager	Tinggi	AWS ecosystem

1. Env File (Development)

```
# .env – di .gitignore!  
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb  
JWT_SECRET=dev-secret-key-not-for-production
```

Penting: - `.env` masuk `.gitignore` - `.env.example` adalah template tanpa nilai rahasia - Jangan pernah commit `.env.production`

2. CI/CD Secrets (GitHub Actions)

```
# .github/workflows/deploy.yml  
name: Deploy  
on:  
  push:  
    branches: [main]  
  
jobs:  
  deploy:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v3  
  
      - name: Deploy to VPS  
        env:  
          DATABASE_URL: ${ secrets.DATABASE_URL }  
          JWT_SECRET: ${ secrets.JWT_SECRET }  
          # Semua env production dari GitHub Secrets  
        run: |  
          echo "DATABASE_URL=$DATABASE_URL" >> .env.production  
          echo "JWT_SECRET=$JWT_SECRET" >> .env.production  
          # deploy script...
```

3. Doppler (Recommended untuk Team)

Setup

```
npm install -g doppler-cli
```

```
doppler setup
```

Jalankan dengan secrets dari Doppler

```
doppler run -- node dist/app.js
```

Sync ke .env lokal

```
doppler secrets download --format env > .env
```

4. HashiCorp Vault

```
import vault from 'node-vault';
```

```
const client = vault({  
  apiVersion: 'v1',  
  endpoint: process.env.VAULT_ADDR || 'http://localhost:8200',  
  token: process.env.VAULT_TOKEN,  
});
```

```
async function getDbCredentials() {  
  const result = await client.read('database/creds/my-role');  
  return {  
    username: result.data.username,  
    password: result.data.password,  
  };  
}
```

Checklist Keamanan Secrets

- ☐ .env di .gitignore
 - ☐ .env.example di-commit
 - ☐ .env.production **tidak** di-commit
 - ☐ Secrets di CI/CD menggunakan built-in secrets manager
 - ☐ Rotasi secrets berkala
 - ☐ Tidak ada hardcode secret di kode
-

4.5 Logging Level Per Environment

Logger dengan Level

```
// logger.ts
type LogLevel = 'error' | 'warn' | 'info' | 'debug';

const LOG_LEVELS: Record<LogLevel, number> = {
  error: 0,
  warn: 1,
  info: 2,
  debug: 3,
};

class Logger {
  private currentLevel: number;

  constructor(level: LogLevel = 'info') {
    this.currentLevel = LOG_LEVELS[level];
  }

  private log(level: LogLevel, message: string, meta?: unknown) {
    if (LOG_LEVELS[level] > this.currentLevel) return;

    const entry = {
      timestamp: new Date().toISOString(),
      level,
      message,
      meta: meta || undefined,
    };

    if (level === 'error') {
      console.error(JSON.stringify(entry));
    } else if (level === 'warn') {
      console.warn(JSON.stringify(entry));
    } else {
      console.log(JSON.stringify(entry));
    }
  }

  error(message: string, meta?: unknown) { this.log('error', message, meta); }
  warn(message: string, meta?: unknown) { this.log('warn', message, meta); }
  info(message: string, meta?: unknown) { this.log('info', message, meta); }
  debug(message: string, meta?: unknown) { this.log('debug', message, meta); }
}
```

```
export const logger = new Logger(process.env.LOG_LEVEL as LogLevel || 'info');
```

Penggunaan

```
import { logger } from './logger';
import { env } from './env';

// Debug – hanya di development
logger.debug('Mencoba koneksi database...', { url: env.DATABASE_URL });

// Info – semua environment
logger.info('Server dimulai', { port: env.PORT, env: env.NODE_ENV });

// Warn – peringatan
logger.warn('Rate limit mendekati batas', { ip: req.ip });

// Error – error yang perlu ditindaklanjuti
logger.error('Koneksi database gagal', { error: err.message });
```

Logging Level Berdasarkan Environment

Environment	Level	Volume	Tujuan
Development	debug	Tinggi	Debugging selama development
Staging	info	Sedang	Verifikasi sebelum production
Production	warn	Rendah	Hanya error dan warning

Struktur Log JSON

```
{"timestamp": "2025-07-05T10:30:00.000Z", "level": "info", "message": "Server dimulai"}
{"timestamp": "2025-07-05T10:30:01.000Z", "level": "error", "message": "Database conn"}
```

Format JSON memudahkan integrasi dengan log aggregation tools (ELK, Datadog, Grafana Loki).

4.6 Feature Flags

Feature flags mengaktifkan/menonaktifkan fitur tanpa deploy ulang.

Berdasarkan Environment Variable

```
import { env } from './env';
```

```

// Cache – aktif di staging & prod
if (env.ENABLE_CACHE) {
  app.use(cacheMiddleware);
}

// Rate limit – aktif di semua kecuali development
if (env.ENABLE_RATE_LIMIT) {
  app.use(rateLimiter);
}

// Fitur beta – hanya di staging
if (env.NODE_ENV === 'staging') {
  app.use('/api/beta', betaFeatureRouter);
}

// Fitur baru – roll out bertahap
const NEW_CHECKOUT_FLOW = process.env.FEATURE_NEW_CHECKOUT === 'true';
if (NEW_CHECKOUT_FLOW) {
  app.post('/api/checkout', newCheckoutHandler);
} else {
  app.post('/api/checkout', oldCheckoutHandler);
}

```

Lebih Terstruktur

```

// features.ts
import { env } from './env';

interface Features {
  cache: boolean;
  rateLimit: boolean;
  newCheckout: boolean;
  betaDashboard: boolean;
  exportCsv: boolean;
}

export const features: Features = {
  cache: env.ENABLE_CACHE,
  rateLimit: env.ENABLE_RATE_LIMIT,
  newCheckout: process.env.FEATURE_NEW_CHECKOUT === 'true',
  betaDashboard: env.NODE_ENV === 'staging',
  exportCsv: env.NODE_ENV !== 'development', // nonaktif di dev
};

// Penggunaan

```

```
if (features.cache) { /* ... */ }
```

4.7 Praktik Lengkap

```
// env.ts
import { z } from 'zod';
import dotenv from 'dotenv';
import path from 'path';

const nodeEnv = process.env.NODE_ENV || 'development';
dotenv.config({ path: path.resolve(process.cwd(), `.env.${nodeEnv}`) });

const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'staging', 'production']),
  PORT: z.coerce.number().int().positive().default(3000),
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url().optional(),
  JWT_SECRET: z.string().min(32),
  JWT_EXPIRES_IN: z.string().default('7d'),
  CORS_ORIGIN: z.string().default('http://localhost:5173'),
  LOG_LEVEL: z.enum(['error', 'warn', 'info', 'debug']).default('info'),
  ENABLE_CACHE: z.coerce.boolean().default(true),
  ENABLE_RATE_LIMIT: z.coerce.boolean().default(true),
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) {
  console.error('❌ Invalid environment variables:');
  parsed.error.issues.forEach(i => console.error(`  ${i.path.join('.')}: ${i.message}`));
  process.exit(1);
}

export const env = parsed.data;
export type Env = z.infer<typeof envSchema>;

# .env.development
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://dev:dev@localhost:5432/mydb_dev
JWT_SECRET=dev-only-secret-key-1234567890123456
LOG_LEVEL=debug
CORS_ORIGIN=http://localhost:5173
ENABLE_CACHE=false
ENABLE_RATE_LIMIT=false
```

```
# .env.production (JANGAN DI-COMMIT!)
NODE_ENV=production
PORT=3000
DATABASE_URL=postgresql://user:pass@production-host:5432/mydb
JWT_SECRET=real-production-secret-that-is-very-long
LOG_LEVEL=warn
CORS_ORIGIN=https://app.example.com
ENABLE_CACHE=true
ENABLE_RATE_LIMIT=true

# .env.example
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
JWT_SECRET=change-this-to-a-secure-key
LOG_LEVEL=info
CORS_ORIGIN=http://localhost:3000
ENABLE_CACHE=true
ENABLE_RATE_LIMIT=true
```

Latihan

Soal 1: Env Validation

Buat file `env.ts` dengan Zod validation untuk environment variables:

1. `NODE_ENV` — enum `'development' | 'staging' | 'production'`, default `'development'`
2. `PORT` — number, 1-65535, default 3000
3. `DATABASE_URL` — string valid URL
4. `JWT_SECRET` — string minimal 32 karakter
5. `SMTP_HOST` — string (opsional)
6. `SMTP_PORT` — number (opsional), 1-65535
7. `LOG_LEVEL` — enum `'error' | 'warn' | 'info' | 'debug'`, default `'info'`
8. `ENABLE_NOTIFICATIONS` — boolean, default false

Jika validasi gagal, tampilkan error dan exit process dengan kode 1.

Soal 2: Multi-Env Config

Buat konfigurasi untuk tiga environment:

1. **Development** (`.env.development`):
 - Local database, debug logging, cache nonaktif
 - CORS origin: `localhost:5173`
2. **Staging** (`.env.staging`):

- Staging database, info logging, cache aktif
 - CORS origin: `https://staging.example.com`
3. **Production** (`.env.production`):
- Production database, warn logging, cache aktif, rate limit aktif
 - CORS origin: `https://example.com`
 - JWT secret: hanya dari environment variable (jangan hard-code!)

Soal 3: Logger + Feature Flags

Buat sistem logging dan feature flags:

1. Logger dengan level: error, warn, info, debug
2. Set level berdasarkan `LOG_LEVEL` dari env
3. Output JSON: `{ timestamp, level, message, meta? }`
4. Fitur feature flags: cache, rate limit, notifikasi
5. Semua flag dikontrol lewat environment variables
6. Contoh penggunaan: middleware cache hanya jalan jika `ENABLE_CACHE=true`